L-Expr: Multiplikation/Division und unäres Minus

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Rückblick: L-Int

- Konkrete Syntax:

```
eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
INT   ::= [0-9]+
lint  ::= eform EOF
```

- Abstrakte Syntax:

```
Expr = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
Lint = Expr
```

- AST:

```
public interface Expr {}
public record AddExpr(Expr left, Expr right) implements Expr {}
public record SubExpr(Expr left, Expr right) implements Expr {}
public record IntLiteral(int value) implements Expr {}
```

Erweiterung zu L-Expr: Multiplikation und Division

- Beispiel:

```
(* 2 (+ 3 4))
```

- **Neue konkrete Syntax in Lispy (Stufe L-Expr)**:

```
eform ::= '(' '+' eform eform+ ')' | '(' '-' eform eform+ ')' | INT
INT   ::= [0-9]+
---

eform ::= '(' '*' eform eform+ ')'
        | '(' '/' eform eform+ ')'
        | '(' '-' eform ')'

lexpr ::= eform EOF
```

Abstrakte Syntax für L-Expr

```
Expr  = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
---

Expr  = MulExpr(Expr, Expr) | DivExpr(Expr, Expr)

Lexpr = Expr
```

```
public interface Expr {}
```

```
public record MulExpr(Expr left, Expr right) implements Expr {}
public record DivExpr(Expr left, Expr right) implements Expr {}
```

Interpreter: Erinnerung an L-Int

```
public class InterpreterLint {

    public Value eval(Expr expr) {
        return switch (expr) {
            case IntLiteral(var v) -> new IntVal(v);

            case AddExpr(var l, var r) -> {
                int a = Value.asInt(eval(l));
                int b = Value.asInt(eval(r));
                yield new IntVal(a + b);
            }

            // TODO: SubExpr

            default -> throw new RuntimeException("unexpected expression: " + expr);
        };
    }
}
```

Interpreter L-Expr: Erben von L-Int und Ergänzen der neuen Funktionalität

```
public class InterpreterLexpr extends InterpreterLint {

    @Override
    public Value eval(Expr expr) {
        return switch (expr) {
            case MulExpr(var l, var r) -> {
                int a = Value.asInt(eval(l));
                int b = Value.asInt(eval(r));
                yield new IntVal(a * b);
            }

            // TODO: DivExpr

            default -> super.eval(expr);
        };
    }
}
```

Wrap-up

- L-Expr = L-Int + Multiplikation, Division und unäres Minus (nur Integer-Ausdrücke)

- Konkrete Syntax: neue Lispy-Formen `(* e1 e2 ...)`, `(/ e1 e2 ...)`, `(- e)`
- Abstrakte Syntax: neue AST-Knoten `MulExpr` und `DivExpr`
- Unäres Minus wird beim AST-Bau auf vorhandene Knoten desugart

- Sprachstufen bauen schrittweise aufeinander auf:
  - Grammatiken übernehmen alte Regeln und ergänzen neue Alternativen
  - Java-Klassen leiten von Vorgängerstufe ab, überschreiben Methoden und nutzen `super.XYZ()`



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.